Day 1 Task: PROBLEM STATEMENT

Design and implement a Leave Management Backend Module for an organization.

This module must:

• Allow employee to apply for leave
• Validate leave request
• Apply business rules
• Store leave records
• Prevent rule violations

REQUIRED PROJECT STRUCTURE

com.organization.leavemanagement

```
├── model
├── service
├── repository
├── util
├── exception
└── main
```

Faculty must follow this strictly.

FUNCTIONAL REQUIREMENTS

Employee Data

Each employee has:

employeeId (String)
name (String)
leaveBalance (int)

Leave Request Data

employeeId
leaveType (CASUAL / SICK / EARNED)
numberOfDays
reason
requestDate

BUSINESS RULES (Enterprise Level)

Leave days must be > 0
Leave days cannot exceed leaveBalance
Reason cannot be null or blank
LeaveType must be valid enum
Maximum 5 consecutive sick leaves
Employee must exist
Deduct leave balance only if validation passes

EXCEPTION DESIGN

Create custom exceptions:

• InvalidLeaveRequestException
• InsufficientLeaveBalanceException
• EmployeeNotFoundException

No returning error strings.

Only exception-driven architecture.

DATA STORAGE

Use:

Option A – ArrayList
Option B – HashMap

Faculty must justify which is better and why.

(Expected: HashMap for O(1) lookup using employeeId)

ENTERPRISE VALIDATION TASK
Implement in ValidationUtil:
• String validation (trim, blank check)
• Enum validation
• Numeric validation
• Defensive programming
ADDITIONAL ENTERPRISE CONSTRAINTS
Service layer:
Must not print
Must not catch exceptions
Must throw upward
Main layer:
Handles exceptions
Displays appropriate message
ADVANCED REQUIREMENT
Add feature:
 Employees cannot apply more than 20 leave days in a year.
Now they must:
• Track total leave usage
• Maintain state
• Apply business rule
This force:
• Data structures
• Condition checks
• Control statements
• Exception handling
• Modular desig

com.organization.leavemanagement

```
├── model
│   ├── LeaveType.java       (enum: CASUAL / SICK / EARNED)
│   ├── Employee.java        (employee entity + annual counter)
│   ├── LeaveRequest.java    (input data object)
│   └── LeaveRecord.java     (approved leave — stored in repo)
├── service
│   └── LeaveService.java    (all business logic, throws upward)
├── repository
│   ├── EmployeeRepository.java  (HashMap — O(1) lookup)
│   └── LeaveRepository.java     (Array List — append + query)
├── util
│   └── ValidationUtil.java  (enterprise defensive validation)
├── exception
│   ├── InvalidLeaveRequestException.java
│   ├── InsufficientLeaveBalanceException.java
│   ├── EmployeeNotFoundException.java
│   └── AnnualLeaveLimitExceededException.java
└── main
    └── LeaveManagementApp.java  (entry point, catches all exceptions)
```

LeaveType.java

```
package com.organization.leavemanagement.model;

public enum LeaveType {
   CASUAL,
   SICK,
   EARNED;

   public static LeaveType fromString(String value) {
      if (value == null) return null;
      try {
         return LeaveType.valueOf(value.trim().toUpperCase());
      } catch (IllegalArgumentException e) {
         return null;
      }
   }
}
```

Employee.java

```
package com.organization.leavemanagement.model;

public class Employee {

   private String employeeId;
   private String name;
   private int leaveBalance;
   private int totalLeaveUsedThisYear;

   public Employee(String employeeId, String name, int leaveBalance) {
      this.employeeId           = employeeId;
      this.name                 = name;
      this.leaveBalance         = leaveBalance;
      this.totalLeaveUsedThisYear = 0;
   }

   public String getEmployeeId()             { return employeeId; }
   public void   setEmployeeId(String id)    { this.employeeId = id; }

   public String getName()                   { return name; }
   public void   setName(String name)        { this.name = name; }

   public int  getLeaveBalance()             { return leaveBalance; }
   public void setLeaveBalance(int balance)  { this.leaveBalance = balance; }

   public int  getTotalLeaveUsedThisYear()           { return totalLeaveUsedThisYear; }
   public void setTotalLeaveUsedThisYear(int total)  { this.totalLeaveUsedThisYear = total;
}

   public void deductLeave(int days) {
      this.leaveBalance         -= days;
```

```
        this.totalLeaveUsedThisYear += days;
    }

    @Override
    public String toString() {
        return "Employee{employeeId='" + employeeId + "', name='" + name
                + "', leaveBalance=" + leaveBalance
                + ", totalLeaveUsedThisYear=" + totalLeaveUsedThisYear + '}';
    }
}
```

LeaveRequest.java

```
package com.organization.leavemanagement.model;

import java.time.LocalDate;

public class LeaveRequest {

    private String    employeeId;
    private LeaveType leaveType;
    private int       numberOfDays;
    private String    reason;
    private LocalDate requestDate;

    public LeaveRequest(String employeeId, LeaveType leaveType,
                        int numberOfDays, String reason, LocalDate requestDate) {
        this.employeeId   = employeeId;
        this.leaveType    = leaveType;
        this.numberOfDays = numberOfDays;
        this.reason       = reason;
        this.requestDate  = requestDate;
    }

    public String    getEmployeeId()   { return employeeId; }
    public LeaveType getLeaveType()    { return leaveType; }
    public int       getNumberOfDays() { return numberOfDays; }
    public String    getReason()       { return reason; }
    public LocalDate getRequestDate()  { return requestDate; }

    public void setEmployeeId(String v)       { this.employeeId   = v; }
    public void setLeaveType(LeaveType v)     { this.leaveType    = v; }
    public void setNumberOfDays(int v)        { this.numberOfDays = v; }
    public void setReason(String v)           { this.reason       = v; }
    public void setRequestDate(LocalDate v)   { this.requestDate  = v; }

    @Override
    public String toString() {
        return "LeaveRequest{employeeId='" + employeeId + "', leaveType=" + leaveType
                + ", numberOfDays=" + numberOfDays + ", reason='" + reason
                + "', requestDate=" + requestDate + '}';
```

```
    }
}
```

LeaveRecord.java

```
package com.organization.leavemanagement.model;

import java.time.LocalDate;

public class LeaveRecord {

    private final String    employeeId;
    private final LeaveType leaveType;
    private final int       numberOfDays;
    private final String    reason;
    private final LocalDate requestDate;
    private final LocalDate approvedDate;

    public LeaveRecord(String employeeId, LeaveType leaveType,
                       int numberOfDays, String reason, LocalDate requestDate) {
        this.employeeId   = employeeId;
        this.leaveType    = leaveType;
        this.numberOfDays = numberOfDays;
        this.reason       = reason;
        this.requestDate  = requestDate;
        this.approvedDate = LocalDate.now();
    }

    public String    getEmployeeId()   { return employeeId; }
    public LeaveType getLeaveType()    { return leaveType; }
    public int       getNumberOfDays() { return numberOfDays; }
    public String    getReason()       { return reason; }
    public LocalDate getRequestDate()  { return requestDate; }
    public LocalDate getApprovedDate() { return approvedDate; }

    @Override
    public String toString() {
        return "LeaveRecord{employeeId='" + employeeId + "', leaveType=" + leaveType
             + ", numberOfDays=" + numberOfDays + ", reason='" + reason
             + "', requestDate=" + requestDate + ", approvedDate=" + approvedDate + '}';
    }
}
```

InvalidLeaveRequestException.java

```
package com.organization.leavemanagement.exception;

public class InvalidLeaveRequestException extends RuntimeException {
    public InvalidLeaveRequestException(String message) { super(message); }
}
```

InsufficientLeaveBalanceException.java

```
package com.organization.leavemanagement.exception;

public class InsufficientLeaveBalanceException extends RuntimeException {
   public InsufficientLeaveBalanceException(String message) { super(message); }
}
```

EmployeeNotFoundException.java

```
package com.organization.leavemanagement.exception;

public class EmployeeNotFoundException extends RuntimeException {
   public EmployeeNotFoundException(String message) { super(message); }
}
```

AnnualLeaveLimitExceededException.java

```
package com.organization.leavemanagement.exception;

public class AnnualLeaveLimitExceededException extends RuntimeException {
   public AnnualLeaveLimitExceededException(String message) { super(message); }
}
```

ValidationUtil.java

```
package com.organization.leavemanagement.util;

import com.organization.leavemanagement.exception.AnnualLeaveLimitExceededException;
import com.organization.leavemanagement.exception.InvalidLeaveRequestException;
import com.organization.leavemanagement.model.LeaveType;

public final class ValidationUtil {

   private ValidationUtil() {}

   public static final int MAX_SICK_CONSECUTIVE_DAYS = 5;
   public static final int MAX_ANNUAL_LEAVE_DAYS     = 20;

   // String — null-safe trim + blank check
   public static void validateString(String value, String fieldName) {
      if (value == null || value.trim().isEmpty()) {
         throw new InvalidLeaveRequestException(
               fieldName + " cannot be null or blank.");
      }
   }

   // Numeric — must be > 0
   public static void validatePositiveInt(int value, String fieldName) {
      if (value <= 0) {
         throw new InvalidLeaveRequestException(
               fieldName + " must be greater than 0. Provided: " + value);
      }
   }
```

```
    // Enum — null means unresolvable string was supplied
    public static void validateLeaveType(LeaveType leaveType) {
        if (leaveType == null) {
            throw new InvalidLeaveRequestException(
                    "LeaveType is invalid. Accepted values: CASUAL, SICK, EARNED.");
        }
    }

    // Business rule — max 5 consecutive sick days
    public static void validateSickLeaveLimit(LeaveType leaveType, int days) {
        if (LeaveType.SICK.equals(leaveType) && days >
MAX_SICK_CONSECUTIVE_DAYS) {
            throw new InvalidLeaveRequestException(
                    "SICK leave cannot exceed " + MAX_SICK_CONSECUTIVE_DAYS
                    + " consecutive days. Requested: " + days);
        }
    }

    // Advanced rule — max 20 days per year
    public static void validateAnnualLimit(int alreadyUsed, int requested, String employeeId)
{
        if (alreadyUsed + requested > MAX_ANNUAL_LEAVE_DAYS) {
            int remaining = MAX_ANNUAL_LEAVE_DAYS - alreadyUsed;
            throw new AnnualLeaveLimitExceededException(
                    "Employee [" + employeeId + "] would exceed the annual limit of "
                    + MAX_ANNUAL_LEAVE_DAYS + " days. Already used: " + alreadyUsed
                    + ", Requested: " + requested + ", Remaining quota: " + remaining);
        }
    }
}
```

EmployeeRepository.java

```
package com.organization.leavemanagement.repository;

import com.organization.leavemanagement.model.Employee;
import java.util.*;

public class EmployeeRepository {

    // HashMap chosen: O(1) lookup/insert by employeeId
    private final Map<String, Employee> employeeStore = new HashMap<>();

    public void save(Employee employee) {
        employeeStore.put(employee.getEmployeeId(), employee);
    }

    public Optional<Employee> findById(String employeeId) {
        return Optional.ofNullable(employeeStore.get(employeeId));
    }
```

```
    public Collection<Employee> findAll()         { return employeeStore.values(); }
    public boolean existsById(String employeeId)   { return
employeeStore.containsKey(employeeId); }
    public int count()                            { return employeeStore.size(); }
}
```

LeaveRepository.java

```
package com.organization.leavemanagement.repository;

import com.organization.leavemanagement.model.LeaveRecord;
import java.util.*;
import java.util.stream.Collectors;

public class LeaveRepository {

    // ArrayList: records are append-only; sequential scan is acceptable
    private final List<LeaveRecord> leaveRecords = new ArrayList<>();

    public void save(LeaveRecord record) { leaveRecords.add(record); }

    public List<LeaveRecord> findByEmployeeId(String employeeId) {
        return leaveRecords.stream()
                .filter(r -> r.getEmployeeId().equals(employeeId))
                .collect(Collectors.toList());
    }

    public List<LeaveRecord> findAll() { return new ArrayList<>(leaveRecords); }
    public int count()                 { return leaveRecords.size(); }
}
```

LeaveService.java

```
package com.organization.leavemanagement.service;

import com.organization.leavemanagement.exception.*;
import com.organization.leavemanagement.model.*;
import com.organization.leavemanagement.repository.*;
import com.organization.leavemanagement.util.ValidationUtil;
import java.util.List;

public class LeaveService {

    private final EmployeeRepository employeeRepository;
    private final LeaveRepository    leaveRepository;

    public LeaveService(EmployeeRepository employeeRepository,
                        LeaveRepository leaveRepository) {
        this.employeeRepository = employeeRepository;
        this.leaveRepository    = leaveRepository;
    }
```

```
    public LeaveRecord applyLeave(LeaveRequest request) {

        // Step 1 — Field-level validation
        ValidationUtil.validateString(request.getEmployeeId(), "Employee ID");
        ValidationUtil.validateLeaveType(request.getLeaveType());
        ValidationUtil.validatePositiveInt(request.getNumberOfDays(), "Number of days");
        ValidationUtil.validateString(request.getReason(), "Reason");
        ValidationUtil.validateSickLeaveLimit(request.getLeaveType(),
request.getNumberOfDays());

        // Step 2 — Employee must exist
        Employee employee = employeeRepository
                .findById(request.getEmployeeId())
                .orElseThrow(() -> new EmployeeNotFoundException(
                        "Employee not found with ID: " + request.getEmployeeId()));

        // Step 3 — Business rules
        if (request.getNumberOfDays() > employee.getLeaveBalance()) {
            throw new InsufficientLeaveBalanceException(
                    "Insufficient leave balance for employee [" + employee.getName() + "]. "
                    + "Requested: " + request.getNumberOfDays()
                    + ", Available: " + employee.getLeaveBalance());
        }

        ValidationUtil.validateAnnualLimit(
                employee.getTotalLeaveUsedThisYear(),
                request.getNumberOfDays(),
                employee.getEmployeeId());

        // Step 4 — Persist and deduct
        employee.deductLeave(request.getNumberOfDays());
        employeeRepository.save(employee);

        LeaveRecord record = new LeaveRecord(
                request.getEmployeeId(), request.getLeaveType(),
                request.getNumberOfDays(), request.getReason(), request.getRequestDate());

        leaveRepository.save(record);
        return record;
    }

    public List<LeaveRecord> getLeaveHistory(String employeeId) {
        ValidationUtil.validateString(employeeId, "Employee ID");
        if (!employeeRepository.existsById(employeeId))
            throw new EmployeeNotFoundException("Employee not found: " + employeeId);
        return leaveRepository.findByEmployeeId(employeeId);
    }

    public int getLeaveBalance(String employeeId) {
```

```
        ValidationUtil.validateString(employeeId, "Employee ID");
        return employeeRepository.findById(employeeId)
                .orElseThrow(() -> new EmployeeNotFoundException(
                        "Employee not found: " + employeeId))
                .getLeaveBalance();
    }

    public int getAnnualLeaveUsed(String employeeId) {
        ValidationUtil.validateString(employeeId, "Employee ID");
        return employeeRepository.findById(employeeId)
                .orElseThrow(() -> new EmployeeNotFoundException(
                        "Employee not found: " + employeeId))
                .getTotalLeaveUsedThisYear();
    }

    public void registerEmployee(Employee employee) {
        ValidationUtil.validateString(employee.getEmployeeId(), "Employee ID");
        ValidationUtil.validateString(employee.getName(), "Employee Name");
        ValidationUtil.validatePositiveInt(employee.getLeaveBalance(), "Leave Balance");
        employeeRepository.save(employee);
    }
}
```

LeaveManagementApp.java

```
package com.organization.leavemanagement.main;

import com.organization.leavemanagement.exception.*;
import com.organization.leavemanagement.model.*;
import com.organization.leavemanagement.repository.*;
import com.organization.leavemanagement.service.LeaveService;
import java.time.LocalDate;
import java.util.List;

public class LeaveManagementApp {

    private static LeaveService leaveService;

    public static void main(String[] args) {
        EmployeeRepository employeeRepo = new EmployeeRepository();
        LeaveRepository    leaveRepo    = new LeaveRepository();
        leaveService = new LeaveService(employeeRepo, leaveRepo);

        seedEmployees();

        testScenario01_ValidCasualLeave();
        testScenario02_ValidSickLeave();
        testScenario03_ValidEarnedLeave();
        testScenario04_InsufficientBalance();
        testScenario05_ExceedSickLeaveLimit();
        testScenario06_EmployeeNotFound();
```

```
    testScenario07_BlankReason();
    testScenario08_ZeroDaysRequested();
    testScenario09_NullLeaveType();
    testScenario10_AnnualLimitExceeded();
    testScenario11_LeaveHistory();
    testScenario12_LeaveBalance();
}

private static void seedEmployees() {
    registerSafe(new Employee("EMP001", "Alice Johnson",  15));
    registerSafe(new Employee("EMP002", "Bob Smith",      8));
    registerSafe(new Employee("EMP003", "Carol Williams", 20));
    // EMP004 intentionally absent for EmployeeNotFound test
}

private static void registerSafe(Employee emp) {
    try { leaveService.registerEmployee(emp); }
    catch (InvalidLeaveRequestException e) {
        System.out.println("Registration failed: " + e.getMessage());
    }
}

private static void testScenario01_ValidCasualLeave() {
    applyAndDisplay(new LeaveRequest("EMP001", LeaveType.CASUAL, 3,
            "Family function", LocalDate.of(2024, 5, 10)));
}
private static void testScenario02_ValidSickLeave() {
    applyAndDisplay(new LeaveRequest("EMP002", LeaveType.SICK, 4,
            "Fever and cold", LocalDate.of(2024, 6, 1)));
}
private static void testScenario03_ValidEarnedLeave() {
    applyAndDisplay(new LeaveRequest("EMP003", LeaveType.EARNED, 7,
            "Annual vacation", LocalDate.of(2024, 7, 15)));
}
private static void testScenario04_InsufficientBalance() {
    applyAndDisplay(new LeaveRequest("EMP002", LeaveType.CASUAL, 10,
            "Extended vacation", LocalDate.of(2024, 8, 1)));
}
private static void testScenario05_ExceedSickLeaveLimit() {
    applyAndDisplay(new LeaveRequest("EMP001", LeaveType.SICK, 6,
            "Prolonged illness", LocalDate.of(2024, 9, 1)));
}
private static void testScenario06_EmployeeNotFound() {
    applyAndDisplay(new LeaveRequest("EMP999", LeaveType.CASUAL, 2,
            "Personal work", LocalDate.of(2024, 10, 1)));
}
private static void testScenario07_BlankReason() {
    applyAndDisplay(new LeaveRequest("EMP001", LeaveType.CASUAL, 1,
            "   ", LocalDate.of(2024, 10, 5)));
}
```

```
    private static void testScenario08_ZeroDaysRequested() {
        applyAndDisplay(new LeaveRequest("EMP001", LeaveType.CASUAL, 0,
                "Half day", LocalDate.of(2024, 10, 6)));
    }
    private static void testScenario09_NullLeaveType() {
        applyAndDisplay(new LeaveRequest("EMP001", LeaveType.fromString("HOLIDAY"),
2,
                "Long weekend", LocalDate.of(2024, 10, 7)));
    }
    private static void testScenario10_AnnualLimitExceeded() {
        // 10 more (total 17) — should succeed
        applyAndDisplay(new LeaveRequest("EMP003", LeaveType.CASUAL, 10,
                "Training trip", LocalDate.of(2024, 11, 1)));
        // 5 more (total 22) — should FAIL
        applyAndDisplay(new LeaveRequest("EMP003", LeaveType.EARNED, 5,
                "Year-end holiday", LocalDate.of(2024, 12, 1)));
    }
    private static void testScenario11_LeaveHistory() {
        try {
            List<LeaveRecord> history = leaveService.getLeaveHistory("EMP001");
            history.forEach(r -> System.out.println("  HISTORY: " + r));
        } catch (EmployeeNotFoundException e) {
            System.out.println("  ERROR: " + e.getMessage());
        }
    }
    private static void testScenario12_LeaveBalance() {
        for (String id : new String[]{"EMP001","EMP002","EMP003"}) {
            try {
                System.out.printf("  %-8s Balance: %2d  Annual Used: %2d/20%n",
                        id,
                        leaveService.getLeaveBalance(id),
                        leaveService.getAnnualLeaveUsed(id));
            } catch (EmployeeNotFoundException e) {
                System.out.println("  ERROR: " + e.getMessage());
            }
        }
    }

    private static void applyAndDisplay(LeaveRequest req) {
        try {
            LeaveRecord record = leaveService.applyLeave(req);
            System.out.println("  APPROVED  : " + record);
        } catch (InvalidLeaveRequestException e) {
            System.out.println("  INVALID REQUEST      : " + e.getMessage());
        } catch (InsufficientLeaveBalanceException e) {
            System.out.println("  INSUFF. BALANCE      : " + e.getMessage());
        } catch (EmployeeNotFoundException e) {
            System.out.println("  EMPLOYEE NOT FOUND   : " + e.getMessage());
        } catch (AnnualLeaveLimitExceededException e) {
            System.out.println("  ANNUAL LIMIT EXCEEDED: " + e.getMessage());
```

```
        }
    }
}
```

Output

```
Registering Employees
   Registered: Employee{employeeId='EMP001', name='Alice Johnson',  leaveBalance=15, totalLeaveUsedThisYear=0}
  Registered: Employee{employeeId='EMP002', name='Bob Smith',      leaveBalance=8, totalLeaveUsedThisYear=0}
  Registered: Employee{employeeId='EMP003', name='Carol Williams', leaveBalance=20, totalLeaveUsedThisYear=0}

Scenario 01 — Valid CASUAL Leave (EMP001, 3 days)
  APPROVED  → LeaveRecord{employeeId='EMP001', leaveType=CASUAL, numberOfDays=3,
              reason='Family function', requestDate=2024-05-10, approvedDate=2026-06-06}

Scenario 02 — Valid SICK Leave (EMP002, 4 days)
   APPROVED  → LeaveRecord{employeeId='EMP002', leaveType=SICK, numberOfDays=4,
              reason='Fever and cold', requestDate=2024-06-01, approvedDate=2026-06-06}

 Scenario 03 — Valid EARNED Leave (EMP003, 7 days)
   APPROVED  → LeaveRecord{employeeId='EMP003', leaveType=EARNED, numberOfDays=7,
              reason='Annual vacation', requestDate=2024-07-15, approvedDate=2026-06-06}

 Scenario 04 — Insufficient Balance (EMP002, 10 days, balance=4 after Sc02)
   INSUFFICIENT BALANCE  : Insufficient leave balance for employee [Bob Smith].
                           Requested: 10, Available: 4

 Scenario 05 — SICK Leave Exceeds 5-Day Limit (EMP001, 6 days)
   INVALID REQUEST       : SICK leave cannot exceed 5 consecutive days. Requested: 6

 Scenario 06 — Employee Not Found (EMP999)
   EMPLOYEE NOT FOUND    : Employee not found with ID: EMP999

 Scenario 07 — Blank Reason (EMP001)
   INVALID REQUEST       : Reason cannot be null or blank.

 Scenario 08 — Zero Days Requested (EMP001)
   INVALID REQUEST       : Number of days must be greater than 0. Provided: 0

 Scenario 09 — Invalid LeaveType (EMP001, type=null)
   INVALID REQUEST       : LeaveType is invalid. Accepted values: CASUAL, SICK, EARNED.

 Scenario 10 — Annual Limit (EMP003): 7 used + 10 + 5 = 22 > 20
  [10a] Applying 10 days (should succeed, total=17)...
```

APPROVED → LeaveRecord{employeeId='EMP003', leaveType=CASUAL, numberOfDays=10,
reason='Training trip', requestDate=2024-11-01, approvedDate=2026-06-06}
[10b] Applying 5 more days (should FAIL, total=22)...
INSUFFICIENT BALANCE : Insufficient leave balance for employee [Carol Williams].
Requested: 5, Available: 3

Scenario 11 — Leave History for EMP001
LeaveRecord{employeeId='EMP001', leaveType=CASUAL, numberOfDays=3,
reason='Family function', requestDate=2024-05-10, approvedDate=2026-06-06}

Scenario 12 — Leave Balance Summary
EMP001 Balance: 12 | Annual Used: 3 / 20
EMP002 Balance: 4 | Annual Used: 4 / 20
EMP003 Balance: 3 | Annual Used: 17 / 20